VJTECH ACADEMY

Client Side Scripting (JavaScript)

Prepared As Per MSBTE I Schema Syllabus

Author:

"Prof. Vishal Jadhav"

[BE in Computer Engineering and Having 10 years of IT industry experience (Worked in Capgemini, Amdocs & currently working in TIBCO Software)]

UNIT 5 – Regular Expression, Rollover and Frames

5.1 Regular Expression – Language of regular expression, Finding non-matching characters, Entering a range of characters, Matching digits and non-digits, Matching punctuations and symbols, Matching words, Replacing a text using regular expressions, Returning the matched characters, Regular expression properties.

5.2 Frames – Create a frame, Invisible borders of frame, calling a child windows, changing a content and focus of child window, writing to a child window, accessing elements of another window.

5.3 Rollover – Creating rollover, text rollover, Multiple actions for rollover, more efficient rollover.

❖ Regular Expression :

Regular expression (regex) is a sequence of characters that define a search pattern. It is used to search, edit, or manipulate text and data. Regex is used in many different programming languages and text editors.

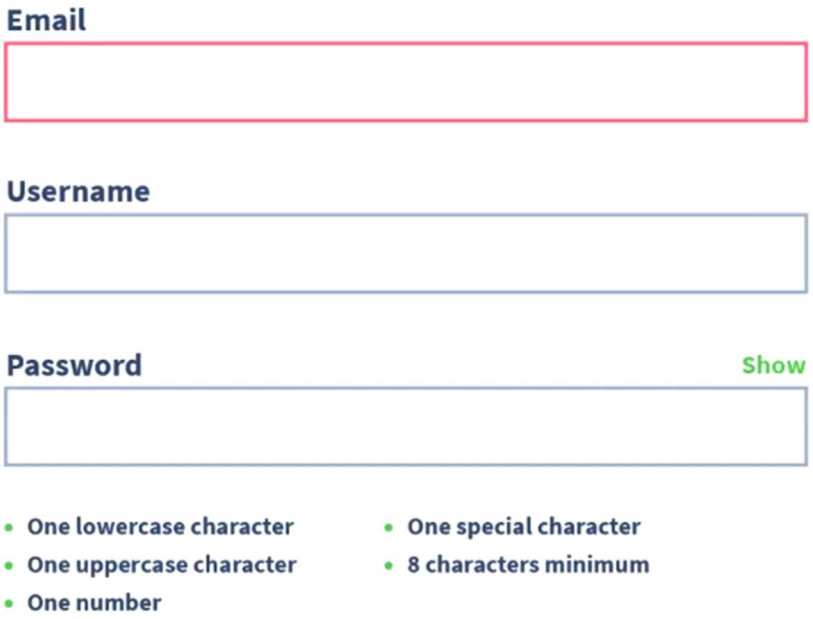
➢ Use Of Regular Expressions :

- Regular expression is a pattern used to match character combinations in strings.
- In JavaScript, regular expressions are also objects.
- These patterns are used with the exec() and test() methods of RegExp, and with the match(), matchAll(), replace(), replaceAll(), search(), and split() methods of String.

➢ Creating a regular expression

- There are two ways to create a regular expression in JavaScript:
  1. Regular expression literal (preferred way) [/pattern/flags].
  2. Regular expression constructor function [new RegExp(pattern[, flags])].

Where we Need RegExp (example)



1. Regular expression literal { recommended }

- In JavaScript, regular expression literals are created by enclosing the pattern in forward slashes (‘ / ’) and optional flags.

1. Syntax:
   - In JavaScript, a regular expression literal has the following syntax: **/pattern/flags**, where:
   - **/ :** Delimiters, indicating the start and end of the regular expression literal.
   - **pattern:** The regular expression pattern you want to match.
   - **flags:** Optional flags that modify how the regular expression behaves.

2. Pattern :
   - The pattern is enclosed between two forward slashes.
   - The pattern can consist of one or more literals, operators, or constructs.
   - For example, /abc/ is a regular expression pattern.
   - The pattern /abc/ matches character combinations in strings only when exactly the characters 'abc' occur together and in that order.

3. Flags :
   - Flags are optional parameters that are used to perform case insensitive and global searches.
   - Flags are specified after the closing slash of the regular expression.
   - There are three flags in JavaScript:
   - **g:** global match
   - **i:** case-insensitive match
   - **m:** multiline match
   - Example with flags:
   - /pattern/g : Searches for all occurrences of pattern in the text.
   - /word/i : Matches "word" regardless of case (e.g., "Word," "WORD," "word").

4. Testing Regular Expressions :
   - You can test regular expressions using methods like **test()** and **match()** on strings in JavaScript to check if a pattern matches or extract matched substrings.

Special Char/Symbols	Description
^	It can match the beginning of a line anywhere within the given expression.
$	It can match the end of a line anywhere within the given expression.
*	Matches **0** or more occurrences of the preceding character or group.
?	Matches **0** or **1** occurrence of the preceding character or group.
+	Matches one or more occurrences of the preceding character or group.
• (dot)	Matches any single character except a newline.
[]	Can be used to specify matching lists
()	Creates a capturing group for a sub-expression.
\b	Can be used for Matching word boundaries
\B	Can be used for Matching non- worded boundaries
{m}	Can be used for matching m times
{m,}	Can be used for matching m times only
{m,n}	Can be used to match at least m times, but for a maximum of n times.

for more info: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions/Cheatsheet

➢ Object Methods Of Regular Expression :

1. "test(string)" method :

- This method checks if a string contains a match for the regular expression pattern.
- It returns **'true'** if a match is found and **'false'** otherwise.
- Example :

```
<script language="javascript" type="text/javascript">
      var pattern = /apple/;
      var text = "I love apples.";
      alert("Your Result Is : "+pattern.test(text)); // Output: true
</script>
```

2. "exec(string)" method :

- exec() searches for a match in the given string and returns an array containing information about the match.
- If no match is found, it returns null.
- The returned array includes the matched text, index, and input string.
- Example :

```
<script language="javascript" type="text/javascript">
      var pattern = /apple/;
      var text = "I love apples and apples are delicious.";
      var result = pattern.exec(text);
      document.write(result);
      // Output: ["apple", index: 7, input: "I love apples and apples are
delicious."]
</script>
```

3. "match(string)" method :

- This method searches for all matches of a regular expression pattern in the given string.
- It returns an array containing all the matched substrings.
- Example :

```
<script language="javascript" type="text/javascript">
     var pattern = /apple/g;
     var text = "I love apples and apples are delicious.";
     var result = text.match(pattern);
     document.write(result); // Output: ["apple", "apple"]
</script>
```

4. "search(string)" method :

- search() searches for a match in the given string and returns the index of the first match. If no match is found, it returns -1.
- Example :

```
<script language="javascript" type="text/javascript">
    var pattern = /apples/;
    var text = "I love apples and apples are delicious.";
    var result  = text.search(pattern);
    alert("First index of apples: " + result);
</script>
```

5. "replace(string, replacement)" method :

- The replace() method allows you to replace matched substrings in a string with a specified replacement string.
- It returns a new string with replacements made.
- Example:

```
<var pattern = /apples/g;
var text = "I love apples and apples are delicious.";
var newText = text.replace(pattern, "oranges");
document.write(newText); //Output:"I love oranges and oranges are delicious"
```

➢ Examples Of Regular Expression :

1. Find non-matching characters in a string using regular expressions

```
<html>
<head>
    <title>Non Matching Character</title>
</head>
<body>
    Enter a string: <input type="text" id="str" />
    <input type="button" value="Find" onclick="find()" />
    <script>
        function find() {
            var str = document.getElementById("str").value;
            var pattern = /[^a-z]/gmi;
            var result = str.match(pattern);
            // returns an array of all the non-matching characters
            alert("Non Matching Character: " + result);
        }
    </script>
</body>
```

✓ Output

In regular expressions, you can define a range of characters using square brackets **“[ ]”** and a hyphen **“-”**. This range allows you to match any character that falls within the specified range.

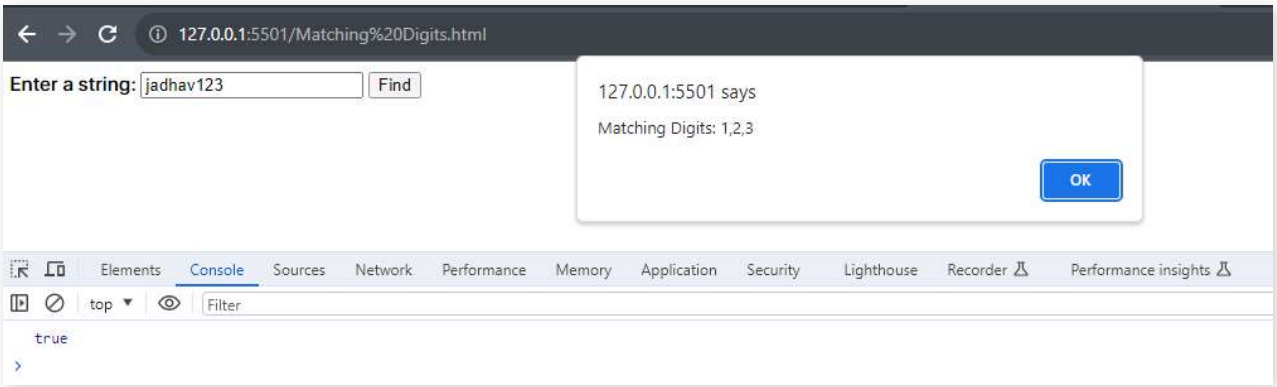
2. Matching Digits or Find Digits

```
<html>
<body>
    Enter a string: <input type="text" id="str" />
    <input type="button" value="Find" onclick="find()" />
    <script language="javascript" type="text/javascript">
        function find() {
            var str = document.getElementById("str").value;
            var pattern = /[0-9]/gmi;

            var result1 = pattern.test(str);
            // returns true if any matching digit is found (way 1)
            console.log(result1);

            // returns an array of all the matching digits (way 2)
            var result2 = str.match(pattern);
            alert("Matching Digits: " + result2);
        }
    </script>
</body>
</html>
```

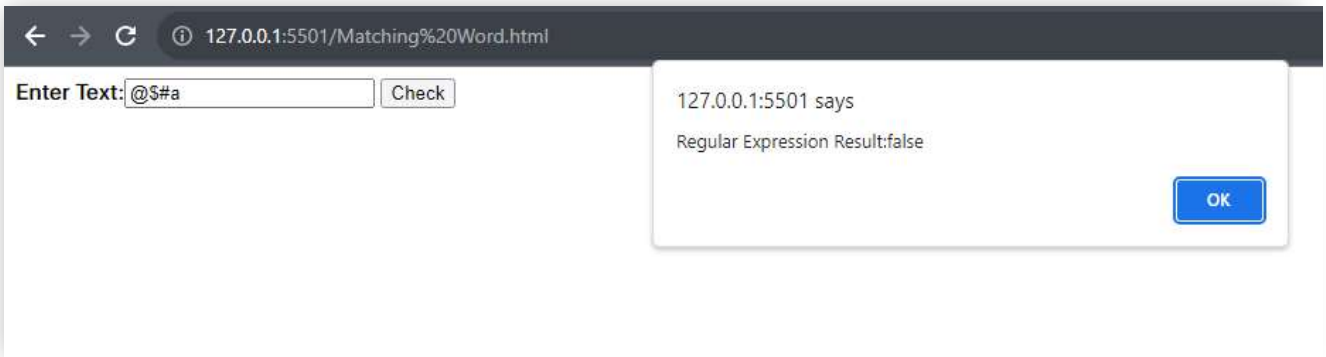
✓ Output



3. Matching Word

```
<html>
<body>
    Enter Text:<input type="text" id="tf1">
    <input type="button" name="b1" value="Check" onclick="CheckOperation()">
    <script language="javascript" type="text/javascript">
        function CheckOperation() {
            var regex = /\w{2}/gmi;
            var str = document.getElementById("tf1").value;
            var result = regex.test(str);
            alert("Regular Expression Result:" + result);
        }
    </script>
</body>
</html>
```

✓ Output



4. Matching Characters

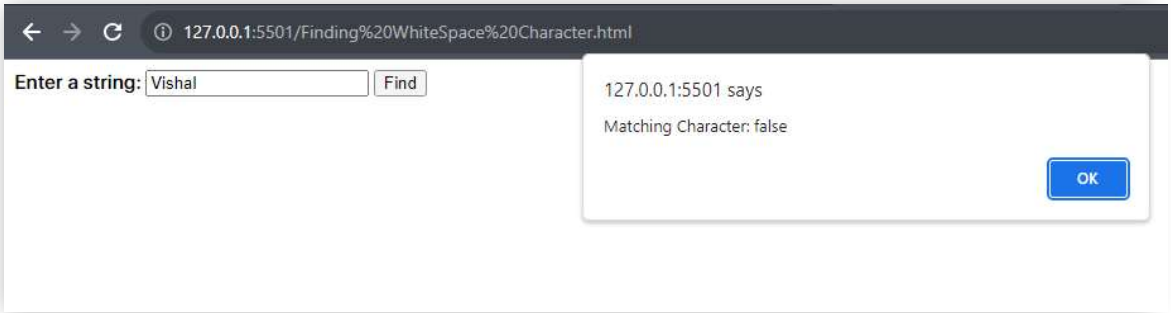
```
<html>
<body>
    Enter a string: <input type="text" id="str" />
    <input type="button" value="Find" onclick="find()" />
    <script language="javascript" type="text/javascript">
        function find() {
            var str = document.getElementById("str").value;
            var pattern = /[abc]/gmi;
            // matches any of the characters a, b or c
            var result = pattern.test(str);
            // returns true if any matching character is found
            alert("Matching Character: " + result);
        }
    </script>
</body>
</html>
```

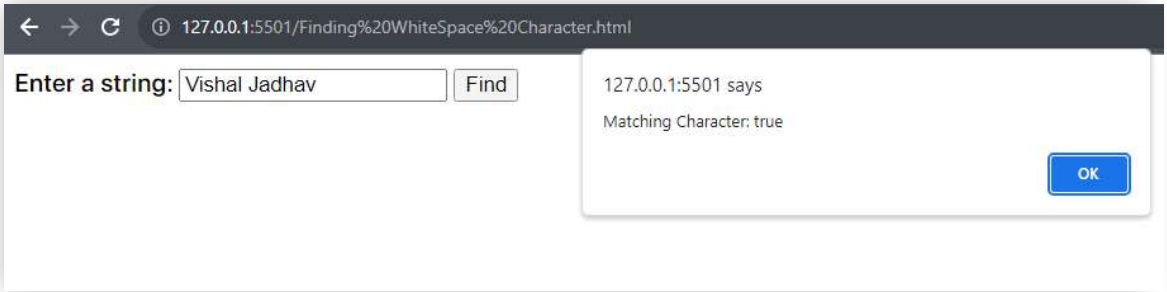
✓ Output

5. Finding/Checking Whitespace Characters

```
<html>
<head>
    <title>Finding/Checking Character</title>
</head>
<body>
    Enter a string: <input type="text" id="str" />
    <input type="button" value="Find" onclick="find()" />
    <script>
        function find() {
            var str = document.getElementById("str").value;
            var pattern = /\s/gmi; // matches any whitespace character
            var result = pattern.test(str);// returns true if any matching
character is found
            alert("Matching Character: " + result);
        }
    </script>
</body>
</html>
```

✓ Output





6. Email Validation :

```
<html>
<head>
    <title>Email Validation</title>
</head>
<body>
   <script language="javascript" type="text/javascript">
        function CheckOperation()
        {
             var regex=/^\w+([-.]?\w+)*@\w+([-.]?\w+)*.\w+([-.]\w+)*$/;
             var str=document.getElementById("tf1").value;
             var result=regex.test(str);
             alert("Regular Expression Result: "+result);
        }
   </script>
   Enter Text:<input type="text" id="tf1">
   <input type="button" name="b1" value="Check" onclick="CheckOperation()">
</body>
</html>
```

7. Mobile No Validation :

```
<html>
<body>
   Enter Your Mobile No :<input type="text" id="tf1">
   <input type="button" name="b1" value="Check" onclick="CheckOperation()">
   <script language="javascript" type="text/javascript">
        function CheckOperation() {
            var regex = /^[0-9]{10}$/;
            var str = document.getElementById("tf1").value;
            var result = regex.test(str);
            if (result == true) {
                alert("Valid Mobile No");
            } else {
                alert("Invalid Mobile No");
            }
        }
    </script>
</body>
</html>
```

Questions For Students :

- What does the '*' quantifier mean in a regular expression?
- How would you match one or more digits in a regular expression?
- How can you perform a case-insensitive search using a regular expression in JavaScript?
- What is the purpose of the [] (square brackets) in a regular expression?
- How do you match any character except a digit using a character class?
- What is the difference between the test() and match() methods for regular expressions in JavaScript?
- How do you replace all occurrences of a pattern in a string using regular expressions?

❖ Frames :

In HTML and JavaScript, frames are a way to divide a web page into multiple independent sections, each of which can display a separate HTML document or web page. Frames were widely used in early web development but have become less common due to various limitations and compatibility issues. Instead, modern web development often relies on other techniques like CSS layouts and iframes. However, it's still useful to understand how frames work in HTML and JavaScript.

1. Creating Frame

- To create frames in HTML, you use the <frameset> element. Inside the <frameset>, you define individual frames using the <frame> or <iframe> elements. The <frameset> element is used to define the layout of frames on a web page.
- Example :

```
<html>
<head>
    <title>Main Frame</title>
</head>
<frameset rows="50%, 50%">
    <frame src="frame1.html">
    <frame src="frame2.html">
</frameset>
</html>
```

HTML Frameset Syntax

1. **“<frameset>”** This tag defines the layout for frames.
   - **rows** attribute : Defines the height of each row of frames.
   - **cols** attribute : Defines the width of each column of frames.
   - Example of frameset with 2 rows

```
<frameset rows="50%, 50%">
   <!-- Frame definitions go here -->
</frameset>
```

2. **<frame>:** Within a <frameset>, you use <frame> tags to define individual frames. Each <frame> tag specifies the source document (src) for the frame.
   - Example:

```
<frameset rows="50%, 50%">
    <frame src="frame1.html">
    <frame src="frame2.html">
</frameset>
```

   - In this example, the web page is divided into two frames, each loading a separate HTML document (**frame1.html** and **frame2.html**).

3. Invisible Frame Border :
   - To create an invisible frame border in HTML, you can use the frameborder attribute in the <frameset> element and set it to "0". This will remove the visible border around the frame, effectively making it invisible.
   - Example

```
<frameset rows="50%, 50%" frameborder="0">
    <frame src="frame1.html">
    <frame src="frame2.html">
</frameset>
```

Frame Examples

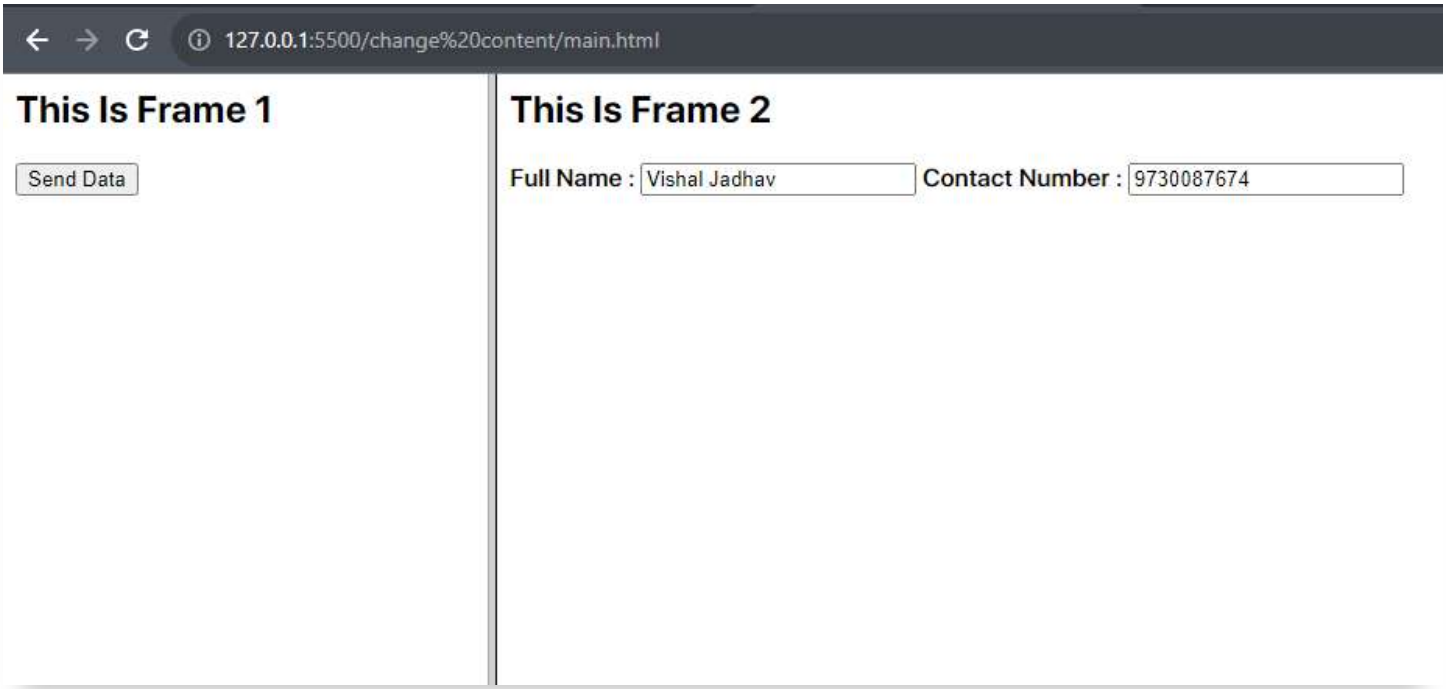
1. Accessing And Changing content of child window

```
<-- Main.html -->
<html>
<head>
    <title>Main Frame</title>
</head>
<frameset cols="50%, 50%">
    <frame src="frame1.html" name="frame1">
    <frame src="frame2.html" name="frame2">
</frameset>
</html>
```

```
<-- Frame1.html -->
<html>
<body>
    <h2>This Is Frame 1</h2>
    <button onclick="display()">Send Data</button>
    <script language="javascript" type="text/javascript">
        function display()
        {
            parent.frame2.document.getElementById("tf1").value="Vishal Jadhav";
            parent.frame2.document.getElementById("tf2").value="9730087674";
            parent.frame2.focus();
        }
   </script>
</body>
</html>
```

```
<-- Frame2.html -->
<<html>
<body>
    <h2>This Is Frame 2</h2>
    Full Name : <input type="text" id="tf1" value="">
    Contact Number : <input type="text" id="tf2" value="">
</body>
</html>
```

✓ **Output :**


127.0.0.1:5500/change%20content/main.html
This Is Frame 1
Send Data
This Is Frame 2
Full Name :
Vishal Jadhav
Contact Number :
9730087674

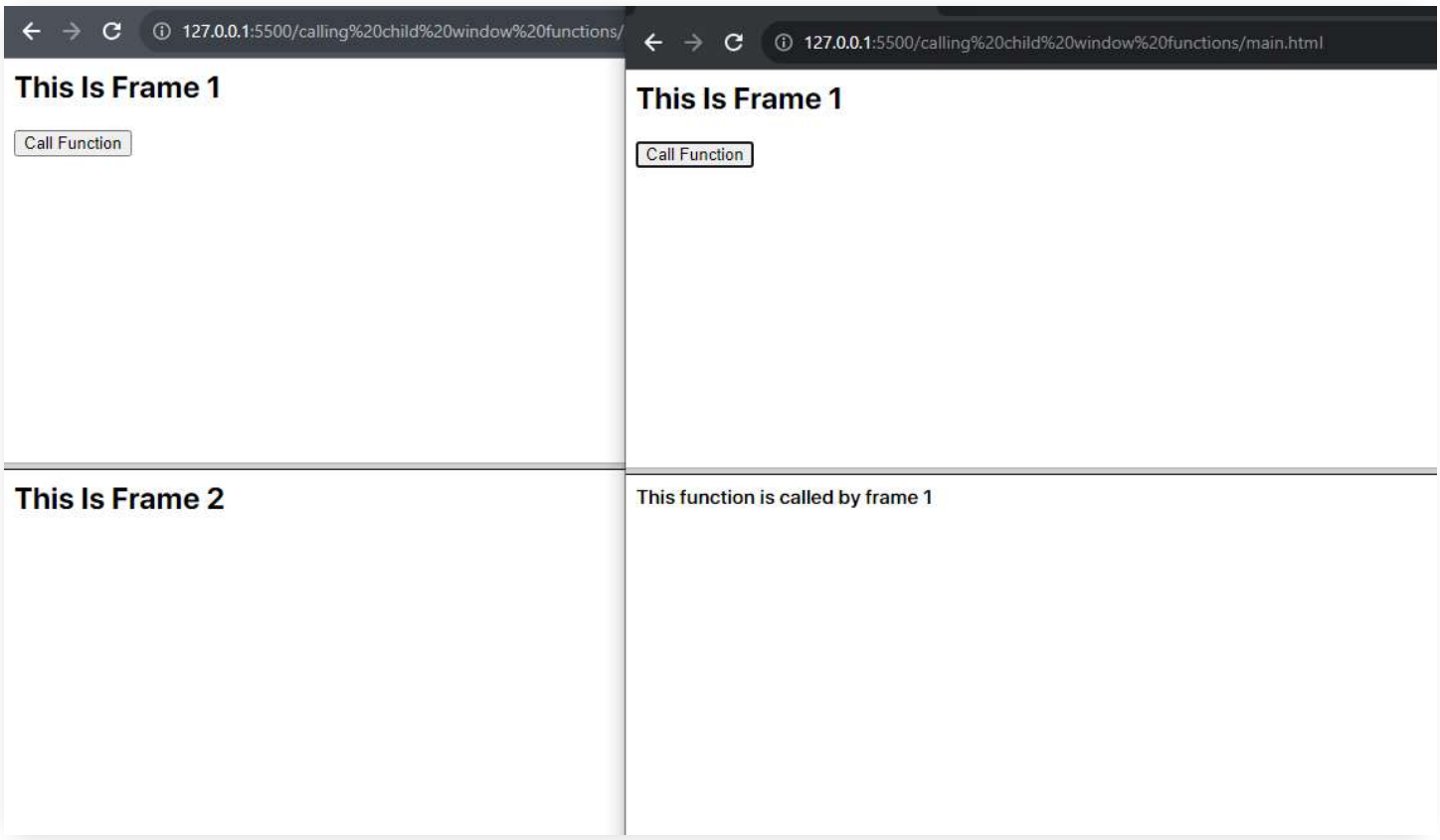
2. Calling child window functions and writing on it.

```
<-- Main.html -->
<html>
<head>
    <title>Main File</title>
</head>
<frameset rows="50%, 50%">
    <frame src="frame1.html" name="frame1">
    <frame src="frame2.html" name="frame2">
</frameset>
</html>
```

```
<-- Frame1.html -->
<html>
<body>
    <h2>This Is Frame 1</h2>
    <input type="button" value="Call Function" onclick="parent.frame2.display()">
</body>
</html>
```

```
<-- Frame2.html -->
<html>
<body>
    <h2>This Is Frame 2</h2>
    <-- This is the function that will be called by frame 1 -->
    <script language="javascript" type="text/javascript">
        function display()
        {
            document.write("This function is called by frame 1");
        }
    </script>
</body>
</html>
```

✓ **Output :**


127.0.0.1:5500/calling%20child%20window%20functions/
This Is Frame 1
Call Function
127.0.0.1:5500/calling%20child%20window%20functions/main.html
This Is Frame 1
Call Function
This Is Frame 2
This function is called by frame 1

3. Changing Dynamically Source Of Frame.

```
<-- Main.html -->
<html>
<head>
    <title>Main File</title>
</head>
<frameset rows="50%, 50%">
    <frame src="frame1.html" name="frame1">
    <frame src="frame2.html" name="frame2">
</frameset>
</html>
```

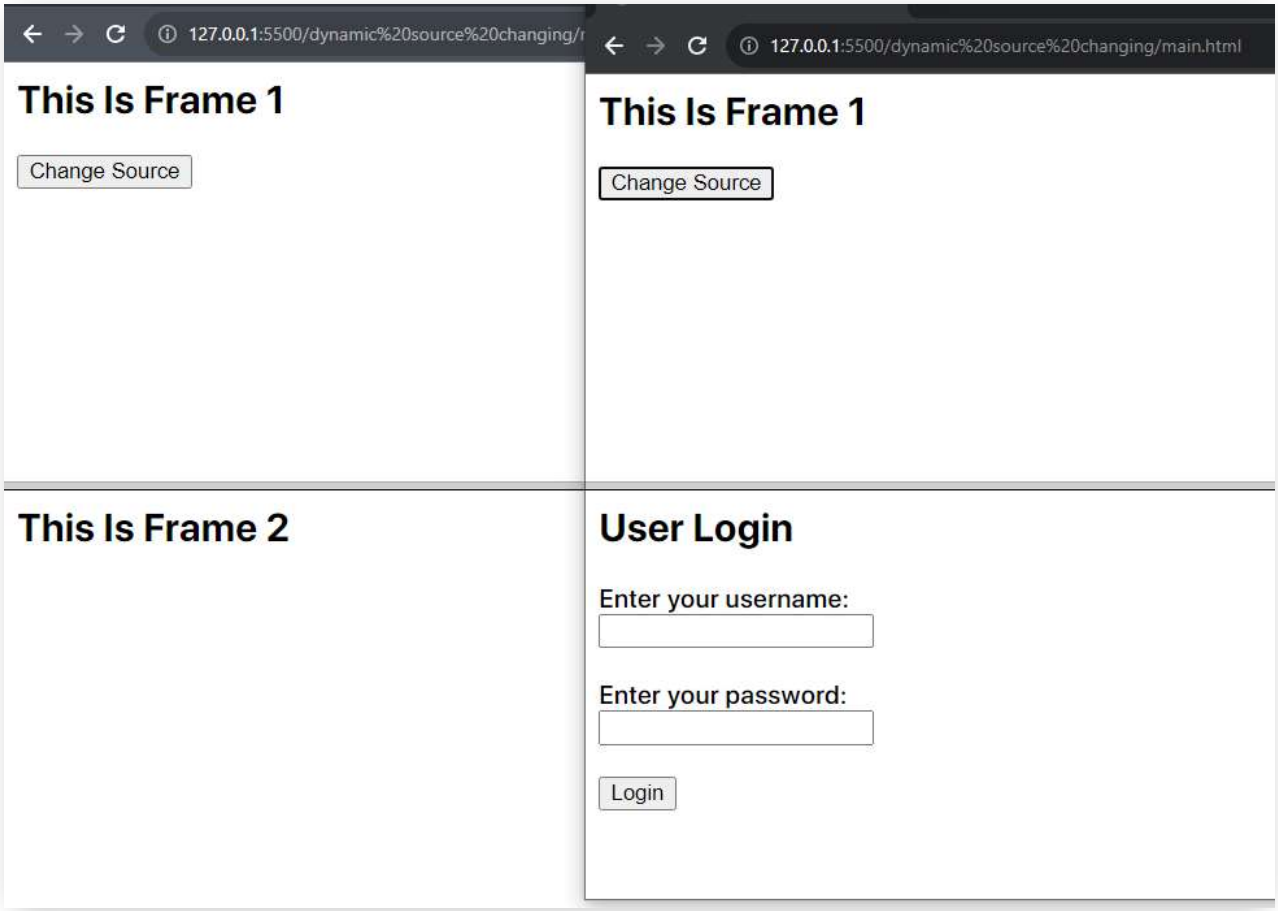
```
<-- Frame1.html -->
<html>
<body>
    <h2>This Is Frame 1</h2>
    <button onclick="changeSource()">Open Login Page</button>

    <script language="javascript" type="text/javascript">
        function changeSource()
        {
            parent.frame2.location.href="login.html";
        }
    </script>
</body>
</html>
```

```
<-- Frame2.html -->
<html>
<body>
    <h2>This Is Frame 2</h2>
</body>
</html>
```

```
<-- login.html -->
<html>
<head>
    <title>login page</title>
</head>
<body>
    <h2>User Login </h2>
    Enter your username: <br><input type="text" id="name"> <br><br>
    Enter your password: <br><input type="password" id="password"> <br><br>
    <input type="button" value="Login">
</body>
</html>
```

✓ **Output :**



❖ Rollover :

- Rollover is a JavaScript/Web technique used by Web developers to produce an effect in which the appearance of a graphical image changes when the user rolls the mouse pointer over it.
- Rollover also refers to a button on a Web page that allows interactivity between the user and the Web page.
- It causes the button to react by either replacing the source image at the button with another image or redirecting it to a different Web page.

1. Creating A Rollover & Rollback

- A rollover is caused by an event called **onmouseover** and occurs when a visitor to your web site moves the mouse over an object that appears on the page.
- An object can be an image, text, or any element of a form. You react to the **onmouseover** event by using the **onmouseover** attribute of an HTML tag that defines the object on the web page and then assign to the onmouseover attribute the action you want performed when the event occurs.
- The action can assign a new value to an attribute of an object, call a method of an object, or call a JavaScript function.
- *Note :"Rollback is same as rollover just difference is **Rollback** occurs when an **"onmouseout"** event trigger's"*
- Example "Image Rollover & Rollback" :

```
<html>
<body>
    <h2>Image RollOver</h2>
    <a><img src="css.png" onmouseover="src='ajava.png'"
onmouseout="src='css.png'" width="500"></a>
</body>
</html>
```

- **Example : Text Rollover :**

```
<html>
<head>
    <title>Text RollOver</title>
</head>
<body>
    <h1>Text RollOver </h1><br>
    <h2>
        <span>--|</span>
        <a onmouseover="document.vjtech.src='html.png'">Html</a>
        <span>|</span>
        <a onmouseover="document.vjtech.src='javascript.png'">JavaScript</a>
        <span>|--</span>
    </h2>
    <br><br>
    <a><img src="vjtech.png" name="vjtech"></a>
</body>
</html>
```

✓ **Output :**

without mouseover

mouseover on JavaScript

- **Example : Multiple Action On Rollover :**

```
<html>
<head>
    <script language="javascript" type="text/javascript">
        function display1() {
            document.vjtech.src = "Apple.jpg";
            document.getElementById("p1").innerHTML = "Apple Image";

        }
        function display2() {
            document.vjtech.src = "PineApple.jpg";
            document.getElementById("p1").innerHTML = "PineApple Image";
        }
    </script>
</head>
<body>
    <h1> Image RollOver </h1><br>
    <h2>
        <span>--|</span>
        <a onmouseover="display1()">Apple</a>
        <span>|</span>
        <a onmouseover="display2()">PineApple</a>
        <span>|--</span>
    </h2><br><br>
    <a><img src="vjtech.png" name="vjtech"></a>
    <h3 id="p1">JavaScript Banner displayed</h3>
</body>
</html>
```

- **Example : More Efficient Rollover:**

```
<html>
<head>
    <title>More Efficient Rollover</title>
</head>
<body>
    <table border="1" width="40%" height="150px">
        <tr>
            <th>Book Name</th>
            <th>Author</th>
            <th>Price</th>
        </tr>
        <tr>
            <td onmouseover="showBook1('Java Programming')">Java Programming</td>
            <td>E Balagurusamy</td>
            <td>200/-</td>
        </tr>
        <tr>
            <td onmouseover="showBook2('C Programming')">C Programming</td>
            <td>Dennis Ritchie</td>
            <td>150/-</td>
        </tr>
    </table>

    <script>
        function showBook1(name) {
            alert("This is : "+name+" Book !!!");
        }

        function showBook2(name) {
            alert("This is : "+name+" Book !!!");
        }
    </script>
</body>
</html>
```

- **Some of the key features of rollover include** *(also home work)* **:**

- Enables interaction between the user and the Web page.
- Makes an image appear or disappear when the mouse is moved over it.
- Makes a hidden image or element to appear when the mouse is moved over it.
- Makes an element of the page change the color of the entire Web page when the mouse is moved over it.
- Causes text to pop up or become highlighted with bold colors when the mouse is moved on a text element.

VJTECH ACADEMY

CLASS NOTES